

Informe exhaustivo de pruebas operativas locales - Dia 3

dealerADMIN | 21 de agosto de 2026 | America/Bogota | C:\dev\dealeradmin | rama main

Resumen ejecutivo

La infraestructura local quedo levantada y comprobada: Docker Desktop, PostgreSQL 18, API NestJS, TypeORM, HMAC, interfaz Next.js, login y dashboard. Los cuatro webhooks respondieron correctamente a nivel HTTP. El criterio de negocio del Dia 3 no fue aprobado porque el servicio actual valida y responde, pero todavia no persiste datos en PostgreSQL.

Area	Resultado
Rama y Git	PASS - main; sin cambios de archivos fuente
Build	PASS
Unitarias	PASS - 2 archivos, 5 pruebas
E2E existente	PASS - 3 de 3
Docker/PostgreSQL	PASS - contenedor y pg_isready
API -> TypeORM -> PostgreSQL	PASS
HMAC	PASS valido e invalido
Escenarios A-D	PASS tecnico - todos 201
Persistencia	FAIL - leads, colas y eventos quedaron en 0
Idempotencia/canonica/mensajes	NOT PROVEN

El informe incluye la cronologia, los fallos intermedios, los comandos, el esquema, el seed, los payloads, las consultas SQL, los resultados, la interfaz y las capturas.

1. Objetivo y reglas

El plan del Dia 3 debia validar idempotencia ante reintentos, una persona canonica en leads con multiples colas independientes en lead_dealers, procesamiento de un lead parcial con telefono, normalizacion telefonica, placeholders de WhatsApp y registro en webhook_events.

- Idempotencia tecnica ante reintentos de GHL.
- Persona canonica unica en leads.
- Multiples relaciones independientes en lead_dealers.
- Lead parcial con telefono aceptado.
- Normalizacion del telefono.
- Mensaje de WhatsApp con placeholders limpios.
- Registro de eventos tecnicos.

2. Rama y estado del repositorio

Al comenzar solo existia codex/dealeradmin-day2; no habia main ni master locales. El arbol estaba limpio. Se renombro directamente la rama actual a main, sin crear otra. El primer intento encontro una restriccion de permisos en el reflog de .git; despues de autorizar la operacion, el renombrado termino. El estado final fue main y git status --short no mostro cambios de archivos fuente.

```
git branch -m main
git branch --show-current
git status --short
```

Resultado: main; sin cambios de archivos fuente.

3. Auditoria inicial

Se revisaron package.json, apps/api/package.json, nest-cli.json, tsconfig.json, main.ts, app.module.ts, WebhookService, InitialSchema, contratos Zod, pruebas unitarias/E2E, .env.example y .gitignore. No habia .env visible y el proceso no heredó variables de entorno. Tampoco habia PostgreSQL en 127.0.0.1:5432. Laragon tenia MySQL, pero no PostgreSQL.

```
NODE_ENV=False
DATABASE_URL=False
JWT_SECRET=False
GHL_WEBHOOK_SECRET=False
ADMIN_USERNAME=False
ADMIN_PASSWORD_HASH=False
PORT=False

5432 open=False
3000 open=False
3001 open=False
```

4. Incidencia del arranque de Nest

La API compilaba sin errores, pero Nest generaba apps/api/dist/apps/api/src/main.js mientras nest start --watch buscaba apps/api/dist/main. El error era Cannot find module C:\dev\dealeradmin\apps\api\dist\main. No se modificó la configuración; se usó el entrypoint generado real.

```
pnpm --filter api build
node apps/api/dist/apps/api/src/main.js
```

El build completo también compiló config, contracts, API y web.

5. Pruebas automatizadas

5.1 Unitarias

```
pnpm test:unit

Test Files  2 passed (2)
Tests       5 passed (5)

lead-eligibility.test.ts: 3
hmac.test.ts: 2
```

5.2 E2E existente

```
pnpm test:e2e

ok 1 - HMAC invalido rechazado
ok 2 - HMAC valido aceptado sobre body exacto
ok 3 - login y dashboard
```

Las 3 pruebas reportaron éxito. El wrapper quedó esperando a sus servidores auxiliares y se detuvo después de conservar el resultado 3/3.

6. Docker Desktop y PostgreSQL

Docker Desktop estaba instalado, pero la primera comprobación no conectaba con docker_engine. Se ejecutó docker desktop start -d; luego docker info confirmó Client/Server 29.7.2, contexto desktop-linux, Docker Desktop sobre WSL2 y arquitectura x86_64.

6.1 Primer intento fallido

Se creó dealeradmin-pgdata y se descargó postgres:18. El primer montaje en /var/lib/postgresql/data hizo que la imagen se detuviera: PostgreSQL 18 requiere montar en /var/lib/postgresql. El contenedor recién creado, vacío, fue retirado.

```
docker volume create dealeradmin-pgdata
docker rm dealeradmin-postgres
```

6.2 Contenedor final

```
docker volume create dealeradmin-pgdata18
docker run --name dealeradmin-postgres -e POSTGRES_USER=dealeradmin -e POSTGRES_PASSWORD=dealeradmin_local -e POSTGRES_DB=dealeradmin -p
```

Contenedor final: c20d4b05ae50. En la recolección final apareció temporalmente Exited (0); se ejecutó docker start dealeradmin-postgres y se volvió a verificar.

```
docker exec dealeradmin-postgres pg_isready -U dealeradmin -d dealeradmin
/var/run/postgresql:5432 - accepting connections

docker ps:
dealeradmin-postgres | Up | 0.0.0.0:5432->5432/tcp
```

Volumenes: dealeradmin-pgdata (primer intento, no utilizado al final) y dealeradmin-pgdata18 (activo).

7. Variables efimeras

No se creo ni se guardo .env. Las variables se inyectaron solo en la sesion del proceso. No se utilizaron credenciales reales de GHL, produccion, Neon ni otro servicio externo.

```
NODE_ENV=development
PORT=3001
DATABASE_URL=postgresql://dealeradmin:dealeradmin_local@127.0.0.1:5432/dealeradmin
JWT_SECRET=local-jwt-secret-change-this-32-characters
GHL_WEBHOOK_SECRET=local-ghl-secret-123456
ADMIN_USERNAME=operator

ADMIN_PASSWORD_HASH: hash Argon2 generado en memoria para local-password.
```

8. Esquema y seed

Se aplico el esquema equivalente a InitialSchema1710000000000: pgcrypto, dealers, leads, lead_dealers, webhook_events y tres indices. Luego se limpiaron tablas y se cargaron los tres dealers del plan.

```
TRUNCATE lead_dealers, leads, webhook_events RESTART IDENTITY CASCADE;
DELETE FROM dealers;
INSERT INTO dealers (code, name, ghl_location_id, timezone, active)
VALUES
('DLR-FRED-001','Offlease Fredericksburg','loc_fredericksburg_123','America/Bogota',true),
('DLR-FRED-002','Offlease Fredericksburg 2','loc_fredericksburg_456','America/Bogota',true),
('DLR-STAFF-001','Offlease Motors Stafford','loc_stafford_789','America/Bogota',true);

CREATE EXTENSION
CREATE TABLE x4
CREATE INDEX x3
TRUNCATE TABLE
DELETE 0
INSERT 0 3
```

9. API conectada

La API se inicio en development con node apps/api/dist/apps/api/src/main.js. Los logs mostraron TypeOrmModule dependencies initialized, TypeOrmCoreModule dependencies initialized, rutas de auth y webhooks, y Nest application successfully started. Esto confirma API -> TypeORM -> PostgreSQL.

10. Escenarios HTTP

Endpoint: POST http://127.0.0.1:3001/api/webhooks. Cada JSON se firmo sobre sus bytes exactos con HMAC-SHA256 y el header X-GHL-Signature: sha256=<firma>.

10.1 A - lead ideal

```
{
  "event_id":"evt_completo_1001",
  "event_type":"lead.ready_for_whatsapp",
  "occurred_at":"2026-08-21T11:00:00Z",
  "dealer_id":"DLR-FRED-001",
  "dealer_name":"Offlease Fredericksburg",
  "ghl_location_id":"loc_fredericksburg_123",
  "ghl_contact_id":"ghl_contact_abc123",
  "lead":{"name":"Carlos Mendoza","phone":"+15551234567","vehicle_type":"SUV","down_payment":"$2,000","purchase_timeline":"Esta semana",}
}

201 {"accepted":true,"eventId":"evt_completo_1001"}
```

10.2 B - lead parcial

```
{
  "event_id":"evt_parcial_2002",
  "event_type":"lead.ready_for_whatsapp",
  "occurred_at":"2026-08-21T11:05:00Z",
  "dealer_id":"DLR-FRED-001",
  "dealer_name":"Offlease Fredericksburg",
  "ghl_location_id":"loc_fredericksburg_123",
  "ghl_contact_id":"ghl_contact_xyz789",
  "lead":{"name":"Maria Lopez","phone":"555-987-6543","vehicle_type":"Sedan","down_payment":null,"purchase_timeline":"","documents":null}
}

201 {"accepted":true,"eventId":"evt_parcial_2002"}
```

10.3 C - reintento duplicado

Se envio exactamente B, con el mismo event_id evt_parcial_2002.

```
201 {"accepted":true,"eventId":"evt_parcial_2002"}
```

10.4 D - multi-dealer

```
{
```

```

    "event_id": "evt_multidealer_3003",
    "event_type": "lead.ready_for_whatsapp",
    "occurred_at": "2026-08-21T11:15:00Z",
    "dealer_id": "DLR-STAFF-001",
    "dealer_name": "Offlease Motors Stafford",
    "ghl_location_id": "loc_stafford_789",
    "ghl_contact_id": "ghl_contact_abc123",
    "lead": { "name": "Carlos Mendoza", "phone": "+15551234567", "vehicle_type": "Troca", "down_payment": "$1,000", "purchase_timeline": "Solo mirando" }
  }
}

201 {"accepted": true, "eventId": "evt_multidealer_3003"}

```

10.5 Firma invalida

```
X-GHL-Signature: sha256=invalid
```

```
401 {"message": "Invalid webhook signature", "error": "Unauthorized", "statusCode": 401}
```

Escenario	HTTP	Respuesta	Evidencia de negocio
A completo	201	accepted evt_completo_1001	No; no persistio
B parcial	201	accepted evt_parcial_2002	No; no persistio
C duplicado	201	accepted evt_parcial_2002	No; no hay deduplicacion
D multi-dealer	201	accepted evt_multidealer_3003	No; no hay cola
Firma invalida	401	Invalid webhook signature	PASS seguridad

11. SQL de validacion

11.1 Conteo

```

SELECT 'dealers', count(*) FROM dealers
UNION ALL SELECT 'leads', count(*) FROM leads
UNION ALL SELECT 'lead_dealers', count(*) FROM lead_dealers
UNION ALL SELECT 'webhook_events', count(*) FROM webhook_events;

```

Tabla	Filas
dealers	3
leads	0
lead_dealers	0
webhook_events	0

11.2 Leads canonicos

```

SELECT id, canonical_phone, first_name, ghl_contact_id FROM leads ORDER BY created_at;
Resultado: (0 rows)

```

11.3 Colas y mensajes

```

SELECT d.name, l.first_name, ld.vehicle_type, ld.status, ld.message_text
FROM lead_dealers ld JOIN leads l ON ld.lead_id=l.id JOIN dealers d ON ld.dealer_id=d.id;
Resultado: (0 rows)

```

11.4 Eventos tecnicos

```

SELECT event_id, event_type, status, processed_at FROM webhook_events;
Resultado: (0 rows)

```

12. Diagnostico definitivo

El problema no es PostgreSQL, HMAC, Zod ni TypeORM. WebhookService.safeParse valida y devuelve {accepted:true,eventId}, pero no usa repositorios, DataSource ni consultas de escritura.

No se inserta webhook_events, no se crea o busca leads, no se normaliza canonical_phone, no se resuelve el dealer, no se inserta lead_dealers, no se deduplican eventos, no se construye message_text y no se registra processed_at.

Regla	Estado
Conexion API -> PostgreSQL	PASS
Contrato y HMAC	PASS
HTTP A-D	PASS tecnico

Regla	Estado
Persistencia	FAIL / no implementada
Idempotencia real	NOT PROVEN
Persona canonica	NOT PROVEN
Colas multi-dealer	NOT PROVEN
Normalizacion	NOT PROVEN
Placeholders	NOT PROVEN
Eventos tecnicos	FAIL / no implementado

13. Interfaz web y capturas

La web se compilo inicialmente sin API_URL y /api/auth/login devolvia 404. Se reconstruyo con API_URL=http://127.0.0.1:3001 y se reinicio Next en 3000. Se comprobo login con operator/local-password, redireccion a /dashboard, cookie de sesion y dashboard.

```
$env:API_URL='http://127.0.0.1:3001'
pnpm --filter web build
$env:PORT='3000'
pnpm --filter web start
```

Estado visible: Session active, Pending leads 0, Active dealers 0 y Webhook status Ready.

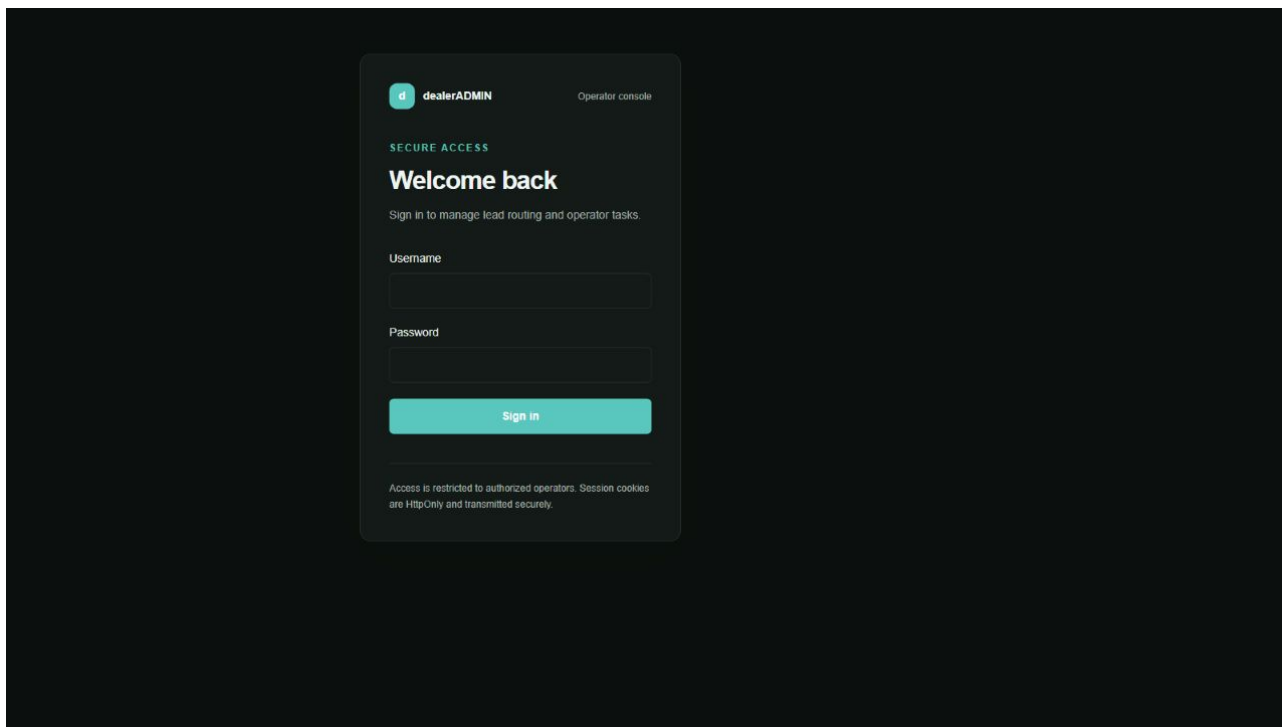


Figura 1. Pantalla de login local.

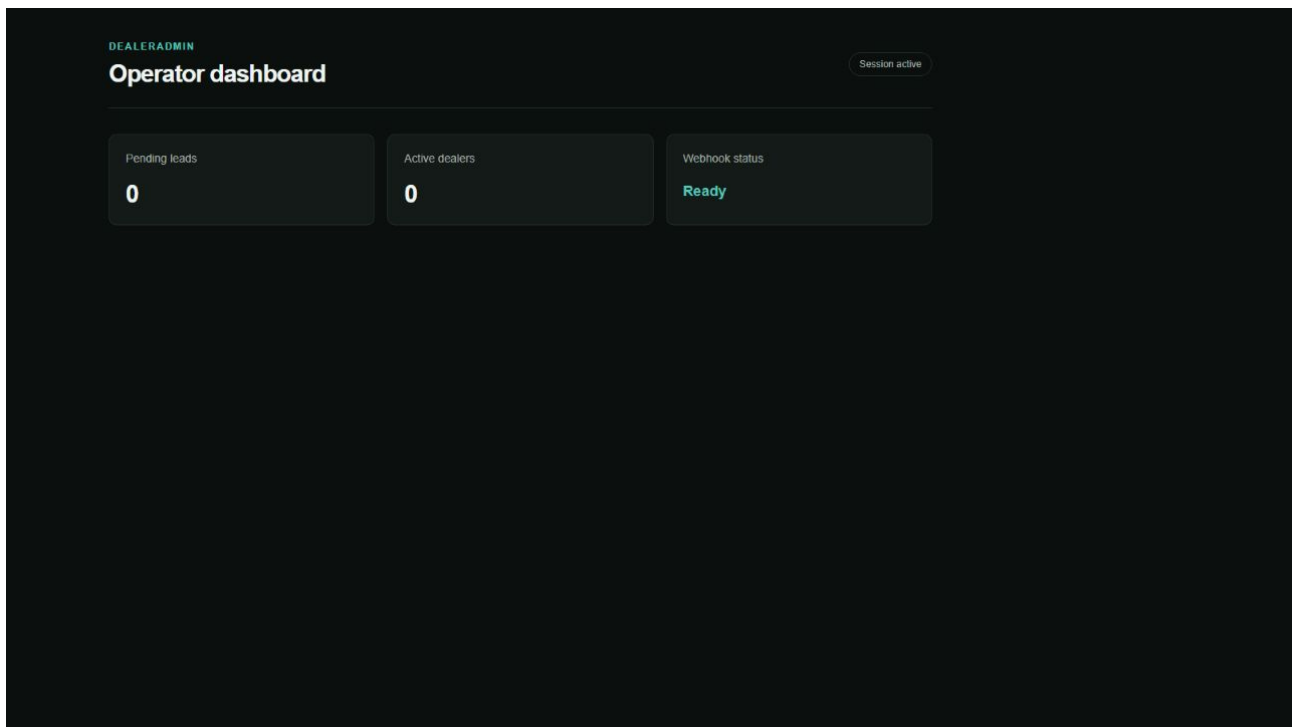


Figura 2. Dashboard local con sesion activa.

Capturas:

C:/Users/Dell/.codex/visualizations/2026/08/21/01a02512-c7cb-7853-b916-8403c4d0f151/dealeradmin-login.png

C:/Users/Dell/.codex/visualizations/2026/08/21/01a02512-c7cb-7853-b916-8403c4d0f151/dealeradmin-dashboard.png

14. Incidencias y acciones realizadas

- Rama unica codex/dealeradmin-day2: renombrada a main.
- Permiso del reflog: autorizado el renombrado puntual.
- Variables y PostgreSQL ausentes: se uso entorno efimero.
- Docker daemon: iniciado/verificado.
- PostgreSQL 18: primer montaje fallido, contenedor vacio retirado y recreado con /var/lib/postgresql.
- Puerto 3001 retenido por proceso anterior PID 15452: proceso cerrado.
- Puerto 3000 retenido por proceso anterior PID 19276: proceso cerrado.
- API reiniciada con hash Argon2 local valido.
- Next reconstruido con API_URL para eliminar el 404 del login.
- Contenedor temporalmente Exited (0): reiniciado y verificado con pg_isready.
- Browser client: se corrigio la ruta interna del cliente y se capturaron login/dashboard.
- No se modificaron archivos fuente ni se utilizaron credenciales reales.

15. Estado final

Componente	Estado
Git	main; arbol limpio
Contenedor	dealeradmin-postgres activo
Imagen	postgres:18
Base/puerto	dealeradmin / 127.0.0.1:5432
Volumen	dealeradmin-pgdata18
API	127.0.0.1:3001

Componente	Estado
Web	127.0.0.1:3000
UI	Dashboard verificado
Credenciales reales	Ninguna

Detener: docker stop dealeradmin-postgres. Iniciar: docker start dealeradmin-postgres.

16. Conclusion

La cadena de infraestructura quedo comprobada hasta el limite actual: payload tipo GHL -> HMAC -> API NestJS -> TypeORM -> PostgreSQL local. La conexion funciona, el esquema existe, los tres dealers fueron cargados, los webhooks se aceptan/rechazan correctamente y la interfaz funciona.

El Dia 3 no puede declararse aprobado todavia.

La evidencia directa es dealers=3, leads=0, lead_dealers=0 y webhook_events=0 despues de A-D. El siguiente trabajo debe implementar persistencia transaccional, normalizacion, deduplicacion, relaciones multi-dealer, estados y formateo de mensajes. Luego se debe repetir esta matriz y verificar 2 leads canonicos, 3 relaciones lead_dealers, 3 eventos y placeholders validos para Maria Lopez.